CHAPTER FOUR

SOFTWARE DESIGN

4.0 Introduction

Chapter Three analysed the requirements of the AI Agent for Real Estate Recommendation and Price Prediction System and modelled its processes and data. This chapter translates that analysis into a design that can be implemented. It presents the architectural design of the system, the design of the user interface, the way in which data are stored, the design of the database, including normalization, the transformation of the entity relationship diagram into relations and the data dictionaries, and the design of the forms and reports of the system.

4.1 Architectural Design

The system follows a three-layer web architecture, shown in Figure 4.1. The presentation layer is the web browser, through which the Public Visitor, the Customer, the Property Manager and the Administrator reach the public portal and the three role-based areas. The application layer is a single Next.js application written in TypeScript. It contains the React user interface, a middleware component that authenticates users and restricts each area to its role, the Route Handlers that provide the backend services, and the three AI modules identified in Chapter Three. The data layer consists of the Prisma ORM, the MySQL database and the file storage that holds property images.

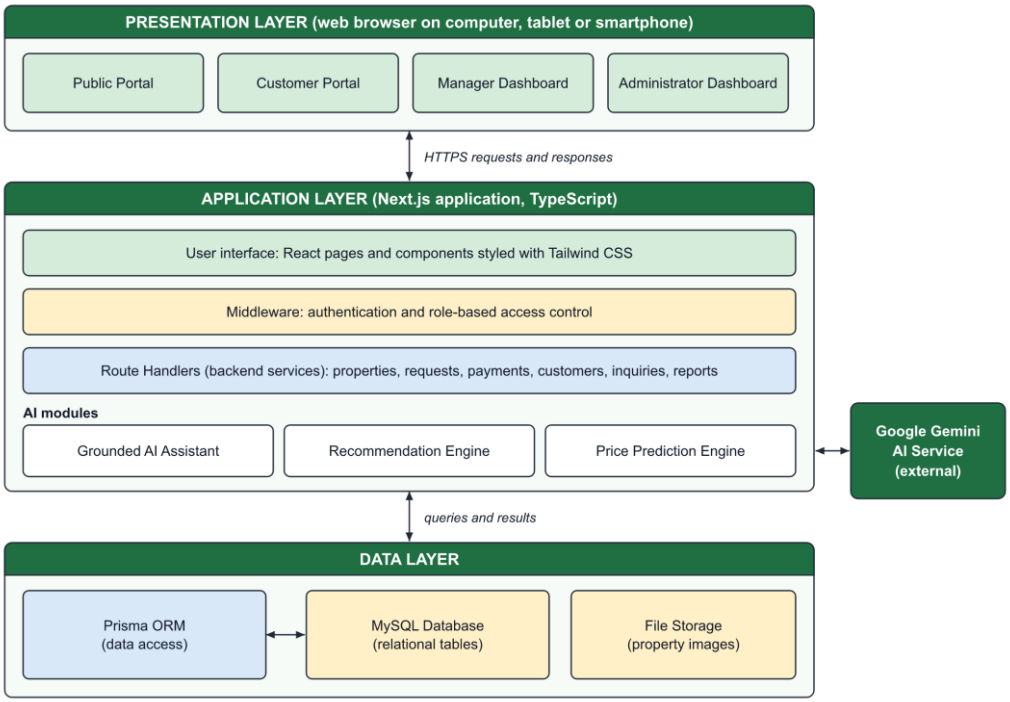


Figure 4.1: Architectural Design of the Proposed System

Table 4.1 summarizes the components of the architecture and the responsibility of each.

Table 4.1: Components of the System Architecture

Component	Layer	Responsibility
Public portal	Presentation	Property search, property details, price prediction and AI assistant for all visitors
Customer portal	Presentation	Favorites, recommendations, rentals, purchases, payments and receipts of one customer
Manager dashboard	Presentation	Properties, customers, rental requests, leases, inquiries and viewings of one manager
Administrator dashboard	Presentation	Users, property approval, sales, ledger, analytics, audit logs and settings
Middleware	Application	Checks the session and the role of the user before a protected page is opened
Route Handlers	Application	Receive requests, validate input, apply the business rules and return data
AI modules	Application	AI assistant, recommendation engine and price prediction engine
Prisma ORM	Data	Maps the application models to tables and executes queries
MySQL database	Data	Stores all structured data of the system
File storage	Data	Stores the image files of properties

This architecture was chosen for three reasons. First, the interface and the backend services share one codebase and one language, which reduces the effort of development and

maintenance. Second, every request passes through the middleware and the Route Handlers before it reaches the database, so the business rules of Chapter Three, such as the rule that only the Property Manager approves rental bookings, are enforced in one place. Third, the AI modules are separated from the business logic: the recommendation and price prediction engines run inside the application, and only the AI assistant communicates with an external service, Google Gemini, to which it sends the question together with the relevant property records.

Security design. Security is built into each layer. In the presentation layer a user sees only the menu of the user's own role. In the application layer the middleware refuses a page that belongs to another role, and each Route Handler checks the role again before it changes data, so a request that bypasses the interface is still refused. In the data layer passwords are stored only as hashes, each e-mail address is unique, and foreign keys prevent a request, payment or receipt from referring to a record that does not exist. Sensitive actions of the Administrator are written to the audit log.

Flow of a request. A typical request passes through the layers as follows. The Customer submits a rental booking in the browser; the middleware confirms that the user is logged in as a Customer; the Route Handler validates the dates, checks that the property is still available and saves the request through Prisma; the database stores the record; and the response returns to the browser, where the new status is displayed. The same path is followed by every operation of the system.

4.2 User Interface Design

The user interface is designed around the four actors of the system. Figure 4.2 shows the navigation structure. All users enter through the home page. Public pages are available without an account, and after login the system redirects each user to the area of the user's role: the customer portal, the manager dashboard or the administrator dashboard. A user cannot open an area that belongs to another role.

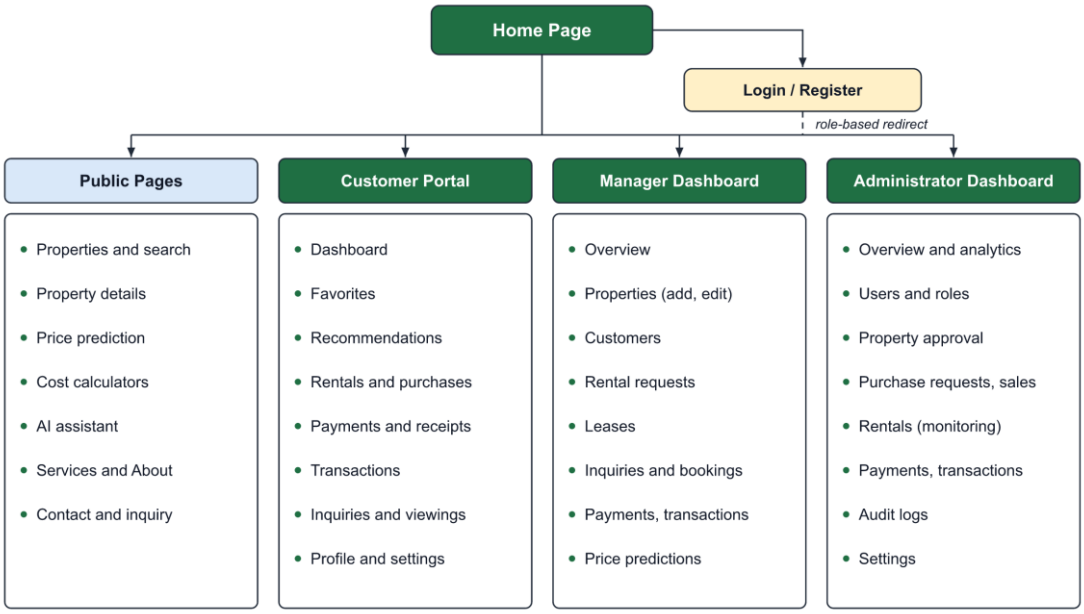


Figure 4.2: Navigation Structure of the User Interface

The interface follows four design principles. Consistency: every page uses the same header, colours, typography and button styles, and the three dashboards share one layout of a sidebar, summary cards and data tables. Simplicity: a visitor can search for a property from the first screen without registering. Responsiveness: the layout adapts to the width of the screen, so tables become cards on small screens. Feedback: the status of every request and payment is displayed with a labelled badge. The interface is presented under the name Kiro-Maal Real Estate. Figure 4.3 shows the implemented home page and the property search interface.

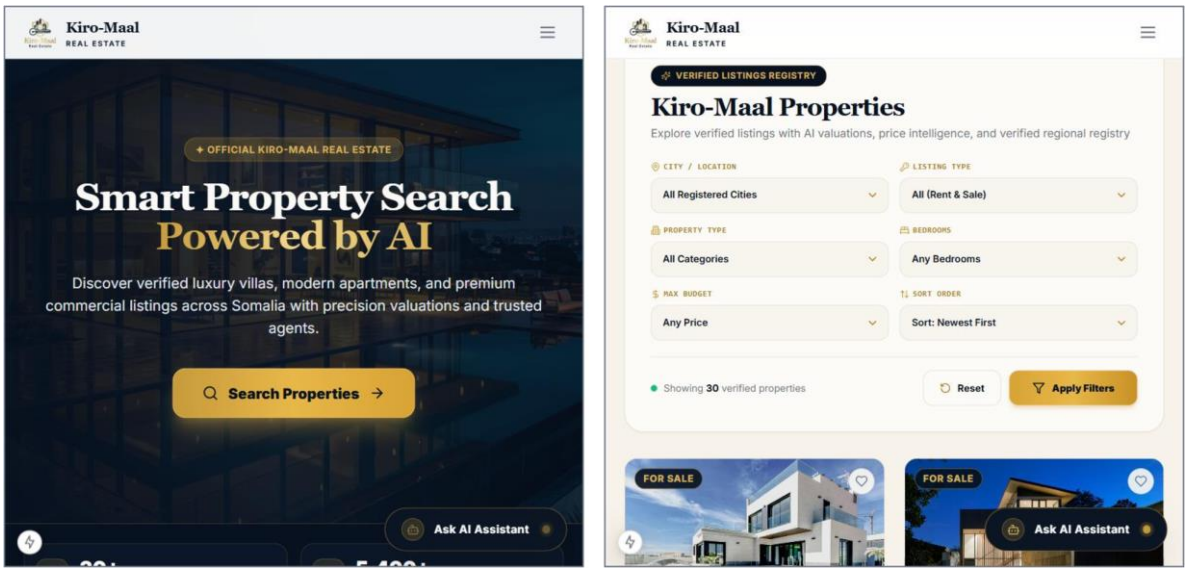


Figure 4.3: Home Page and Property Search Interface

Table 4.2 lists the main screens of the interface, the actor who uses each of them and its purpose.

Table 4.2: Main Screens of the User Interface

Screen	Actor	Purpose
Home page	All users	Entry point with the property search button and the AI assistant
Properties	Public Visitor, Customer	List of published properties with filters and sorting
Property details	Public Visitor, Customer	Specifications, amenities, location, status and actions to rent, buy or inquire
Price prediction	Public Visitor, Customer	Form for the features of a property and the resulting estimate
Login and registration	Registered actors	Sign-in and creation of a customer account
Customer dashboard	Customer	Summary of active leases, pending purchases, saved properties and inquiries
Recommendations	Customer	Properties suggested by the recommendation engine with reasons
Manager overview	Property Manager	Summary cards, today's appointments and pending rental requests
Add property	Property Manager	Multi-step form for details, pricing, images, location and amenities
Rental management	Property Manager	Review of bookings, direct rental creation and lease tracking
Property approval	Administrator	Review of pending properties with approve, reject and publish actions
Requests and sales	Administrator	Confirmation of purchase requests and approval of sales
Transactions	Manager, Administrator	Central ledger with filters by payment method and status

4.3 Database Storage Design

The data of the system are stored in a MySQL relational database. Each entity of the ERD in Chapter Three is stored as one table, each record as one row, and each relationship as a foreign key. The tables are created and changed through the Prisma schema, which keeps the structure of the database and the application code consistent. Property images are not stored inside the database; the image files are kept in the file storage of the application, and only their addresses are stored in the PROPERTY_IMAGES table, which keeps the database small and queries fast. Passwords are never stored in readable form; only their hashes are stored. Table 4.3 lists the data types used.

Table 4.3: Data Types Used in the Database

Data Type	Use in the Proposed System
VARCHAR	Identifiers, names, e-mail addresses, cities, request and receipt numbers
TEXT	Long text such as property descriptions, messages and review notes

Data Type	Use in the Proposed System
INT	Whole numbers such as bedrooms, bathrooms and view counts
DOUBLE	Prices, rent, deposits, areas and geographic coordinates
BOOLEAN	Yes or no values such as is_active, is_furnished and is_read
DATETIME	Dates and times such as created_at, start_date, end_date and paid_at
ENUM	Fixed lists of values such as role, property type and status

4.4 Database Design

The database design converts the conceptual model of Chapter Three into tables that are free from redundancy. This section explains normalization, transforms the ERD into relations and defines the data dictionaries.

4.4.1 Database Normalization

Normalization is the process of organizing data into tables so that each fact is stored once and update anomalies are avoided (Elmasri & Navathe, 2016). It is illustrated below with the rental data of the system, beginning with an unnormalized record in which all the details of a rental are written together: RENTAL (request_no, customer_name, customer_phone, property_title, property_city, manager_name, start_date, end_date, rent_amount, payment_1, payment_2, ...).

4.4.1.1 First Normal Form (1NF)

A table is in first normal form when every field holds a single value and there are no repeating groups. In the unnormalized record the payments form a repeating group. They are therefore moved to a separate table in which each payment is one row: RENTAL (request_no, customer_name, customer_phone, property_title, property_city, manager_name, start_date, end_date, rent_amount) and PAYMENT (payment_id, request_no, amount, payment_method, paid_at).

4.4.1.2 Second Normal Form (2NF)

A table is in second normal form when it is in 1NF and every non-key field depends on the whole primary key. Each table of the system has a single-field primary key, such as rental_request_id, so no field can depend on only part of a key. Junction data are treated in the same way: FAVORITES has its own key, favorite_id, and stores only the two foreign keys and the date.

4.4.1.3 Third Normal Form (3NF)

A table is in third normal form when it is in 2NF and no non-key field depends on another non-key field. In the rental record, customer_phone depends on the customer and property_city depends on the property, not on the rental. These fields are therefore removed to the tables to which they belong, and the rental keeps only foreign keys: USERS (user_id, name, phone, ...), PROPERTIES (property_id, title, city, ...) and RENTAL_REQUESTS (rental_request_id, request_no, customer_id, property_id, manager_id, start_date, end_date, rent_amount, status).

Table 4.4 summarizes the three normal forms and the way in which each is satisfied in the proposed system.

Table 4.4: Summary of the Normal Forms

Form	Rule	Application in the Proposed System
1NF	Every field holds one value; no repeating groups	Payments and property images are stored as separate rows in their own tables
2NF	Every non-key field depends on the whole primary key	Every table has a single-field primary key; junction tables hold only keys and their own data
3NF	No non-key field depends on another non-key field	Customer, manager and property details are stored once and reached through foreign keys

Normalization gives the system three practical benefits. A change, such as a new telephone number of a customer, is made in one place. A record cannot be lost by accident, because deleting a payment does not delete the customer or the property. And the size of the database stays small, because the same text is not repeated in every request.

4.4.1.4 Normalized Tables of the Proposed System

Applying the same steps to all the data of the system produces the normalized tables listed in Table 4.5.

Table 4.5: Normalized Tables of the Proposed System

Table	Primary Key	Foreign Keys
users	user_id	None
properties	property_id	manager_id
property_images	image_id	property_id
favorites	favorite_id	user_id, property_id
search_history	search_id	user_id
recommendations	recommendation_id	user_id, property_id
price_predictions	prediction_id	user_id, property_id
rental_requests	rental_request_id	customer_id, property_id, manager_id
purchase_requests	purchase_request_id	customer_id, property_id, reviewed_by

Table	Primary Key	Foreign Keys
transactions	transaction_id	customer_id, property_id, rental_request_id, purchase_request_id
payments	payment_id	transaction_id, customer_id
receipts	receipt_id	payment_id, transaction_id
inquiries	inquiry_id	property_id, customer_id, manager_id
appointments	appointment_id	property_id, customer_id, manager_id
reviews	review_id	property_id, user_id
notifications	notification_id	user_id
audit_logs	log_id	user_id

4.4.2 Transforming E-R Diagrams into Relations

The ERD of Chapter Three is transformed into relations by three rules. Each entity becomes a relation whose primary key is the identifier of the entity. Each one-to-many relationship is represented by placing the primary key of the "one" side as a foreign key in the relation of the "many" side. Each many-to-many relationship becomes a relation of its own that contains the keys of both sides. Applying these rules gives the following main relations, in which the primary key is written first and foreign keys are marked (FK).

USERS (user_id, name, email, phone, password_hash, role, is_active, created_at)

PROPERTIES (property_id, manager_id (FK), title, description, type, listing_type, status, price, city, location, bedrooms, bathrooms, area, amenities)

FAVORITES (favorite_id, user_id (FK), property_id (FK), created_at)

RENTAL_REQUESTS (rental_request_id, request_no, customer_id (FK), property_id (FK), manager_id (FK), start_date, end_date, rent_amount, security_deposit, status)

PURCHASE_REQUESTS (purchase_request_id, request_no, customer_id (FK), property_id (FK), reviewed_by (FK), sale_price, status, reviewed_at, completed_at)

TRANSACTIONS (transaction_id, txn_no, type, customer_id (FK), property_id (FK), rental_request_id (FK), purchase_request_id (FK), amount, status)

PAYMENTS (payment_id, transaction_id (FK), customer_id (FK), amount, payment_method, status, transaction_ref, paid_at)

RECEIPTS (receipt_id, receipt_no, payment_id (FK), transaction_id (FK), issued_at)

4.4.3 Data Dictionaries

A data dictionary describes each field of a table by its data type, its constraint and its meaning. Tables 4.6 to 4.11 present the data dictionaries of the six central tables of the system. The field names follow the ERD of Chapter Three; identifiers are stored as text keys generated by the system.

Table 4.6: Data Dictionary of the Users Table

Field	Data Type	Constraint	Description
user_id	VARCHAR	Primary key	Unique identifier of the user
name	VARCHAR	Not null	Full name
email	VARCHAR	Unique, not null	E-mail address used to log in
phone	VARCHAR	Optional	Telephone number
password_hash	VARCHAR	Not null	Hashed password
role	ENUM	Not null	Administrator, Property Manager or Customer
is_active	BOOLEAN	Default true	Active or suspended account
created_at	DATETIME	Not null	Date of registration

Table 4.7: Data Dictionary of the Properties Table

Field	Data Type	Constraint	Description
property_id	VARCHAR	Primary key	Unique identifier of the property
manager_id	VARCHAR	Foreign key	Property Manager assigned to the property
title	VARCHAR	Not null	Title of the listing
description	TEXT	Not null	Description of the property
type	ENUM	Not null	House, apartment, villa, office, land and others
listing_type	VARCHAR	Not null	For sale or for rent
status	ENUM	Default pending	Draft, pending, approved, published, rented, sold, archived
price	DOUBLE	Not null	Sale price or rent
city, location	VARCHAR	Not null	City and district of the property
bedrooms, bathrooms	INT	Not null	Number of bedrooms and bathrooms
area	DOUBLE	Not null	Area in square metres
amenities	TEXT	Optional	List of amenities

Table 4.8: Data Dictionary of the Rental Requests Table

Field	Data Type	Constraint	Description
rental_request_id	VARCHAR	Primary key	Unique identifier of the rental request

Field	Data Type	Constraint	Description
request_no	VARCHAR	Unique	Reference number shown to users
customer_id	VARCHAR	Foreign key	Customer who rents
property_id	VARCHAR	Foreign key	Property that is rented
manager_id	VARCHAR	Foreign key	Property Manager who decides on the request
start_date, end_date	DATETIME	Not null	Rental period
rent_amount	DOUBLE	Not null	Rent for one period
security_deposit	DOUBLE	Not null	Deposit paid by the customer
status	ENUM	Default pending	Pending, approved, rejected, active, expired, cancelled

Table 4.9: Data Dictionary of the Purchase Requests Table

Field	Data Type	Constraint	Description
purchase_request_id	VARCHAR	Primary key	Unique identifier of the purchase request
request_no	VARCHAR	Unique	Reference number shown to users
customer_id	VARCHAR	Foreign key	Customer who buys
property_id	VARCHAR	Foreign key	Property that is bought
sale_price	DOUBLE	Not null	Agreed sale price
status	ENUM	Default pending	Pending, payment pending, approved, completed, rejected
reviewed_by	VARCHAR	Foreign key	Administrator who reviewed the request
reviewed_at, completed_at	DATETIME	Optional	Dates of review and completion

Table 4.10: Data Dictionary of the Transactions Table

Field	Data Type	Constraint	Description
transaction_id	VARCHAR	Primary key	Unique identifier of the transaction
txn_no	VARCHAR	Unique	Transaction number
type	ENUM	Not null	Sale or rental
customer_id	VARCHAR	Foreign key	Customer of the transaction
property_id	VARCHAR	Foreign key	Property of the transaction
rental_request_id	VARCHAR	Foreign key, unique	Related rental request, if any
purchase_request_id	VARCHAR	Foreign key, unique	Related purchase request, if any
amount	DOUBLE	Not null	Amount of the transaction
status	ENUM	Default pending	Pending, paid, completed, active, rejected, cancelled

Table 4.11: Data Dictionary of the Payments Table

Field	Data Type	Constraint	Description
payment_id	VARCHAR	Primary key	Unique identifier of the payment
transaction_id	VARCHAR	Foreign key	Transaction settled by the payment
customer_id	VARCHAR	Foreign key	Customer who paid
amount	DOUBLE	Not null	Amount paid
payment_method	VARCHAR	Not null	Card, bank wire, cash or mobile money
status	ENUM	Not null	Pending, paid, failed or refunded
transaction_ref	VARCHAR	Unique	Payment reference
paid_at	DATETIME	Optional	Date and time of payment

Table 4.12: Data Dictionary of the Property Images Table

Field	Data Type	Constraint	Description
image_id	VARCHAR	Primary key	Unique identifier of the image
property_id	VARCHAR	Foreign key	Property to which the image belongs
url	VARCHAR	Not null	Address of the image file in the file storage
is_primary	BOOLEAN	Default false	Marks the main image of the property
sort_order	INT	Default 0	Position of the image in the gallery

Table 4.13: Data Dictionary of the Favorites Table

Field	Data Type	Constraint	Description
favorite_id	VARCHAR	Primary key	Unique identifier of the saved item
user_id	VARCHAR	Foreign key	Customer who saved the property
property_id	VARCHAR	Foreign key	Property that was saved
created_at	DATETIME	Not null	Date on which the property was saved

Table 4.14: Data Dictionary of the Recommendations Table

Field	Data Type	Constraint	Description
recommendation_id	VARCHAR	Primary key	Unique identifier of the recommendation
user_id	VARCHAR	Foreign key	Customer who receives the recommendation
property_id	VARCHAR	Foreign key	Recommended property
score	DOUBLE	Not null	Matching score from 0 to 100
reasons	TEXT	Not null	Reasons shown to the customer

Field	Data Type	Constraint	Description
is_viewed	BOOLEAN	Default false	Whether the customer has seen it

Table 4.15: Data Dictionary of the Price Predictions Table

Field	Data Type	Constraint	Description
prediction_id	VARCHAR	Primary key	Unique identifier of the prediction
user_id	VARCHAR	Foreign key	User who requested the prediction
property_id	VARCHAR	Foreign key, optional	Property concerned, if any
location	VARCHAR	Not null	City or district entered
property_type	ENUM	Not null	Type of the property
area, bedrooms, bathrooms	DOUBLE, INT	Not null	Features entered by the user
predicted_price	DOUBLE	Not null	Estimated market value
min_price, max_price	DOUBLE	Not null	Lower and upper range of the estimate

Table 4.16: Data Dictionary of the Receipts Table

Field	Data Type	Constraint	Description
receipt_id	VARCHAR	Primary key	Unique identifier of the receipt
receipt_no	VARCHAR	Unique	Receipt number shown to users
payment_id	VARCHAR	Foreign key, unique	Payment for which the receipt is issued
transaction_id	VARCHAR	Foreign key, unique	Transaction of the receipt
details	TEXT	Not null	Itemized content of the receipt
issued_at	DATETIME	Not null	Date of issue

Table 4.17: Data Dictionary of the Inquiries Table

Field	Data Type	Constraint	Description
inquiry_id	VARCHAR	Primary key	Unique identifier of the inquiry
property_id	VARCHAR	Foreign key	Property about which the question is asked
customer_id	VARCHAR	Foreign key	User who sent the inquiry
manager_id	VARCHAR	Foreign key	Property Manager who answers
message	TEXT	Not null	Text of the inquiry
response	TEXT	Optional	Answer of the manager
status	ENUM	Default new	New, read, responded or closed

The remaining tables, namely appointments, reviews, search history, notifications and audit logs, follow the same pattern: a generated primary key, the foreign keys shown in Table 4.5, their own descriptive fields and the date of creation.

4.5 Designing Forms and Reports

Forms are the means by which users enter data, and reports are the means by which the system presents stored data. The forms of the system follow common rules: related fields are grouped, required fields are marked, lists are used wherever the possible values are fixed, such as city and property type, and each form is validated before it is saved, with the error shown beside the field. The main forms are the login and registration form, the multi-step property form of the Property Manager, the rental booking form, the purchase request form, the payment form, the inquiry form and the price prediction form. Figure 4.4 shows two of the implemented forms.

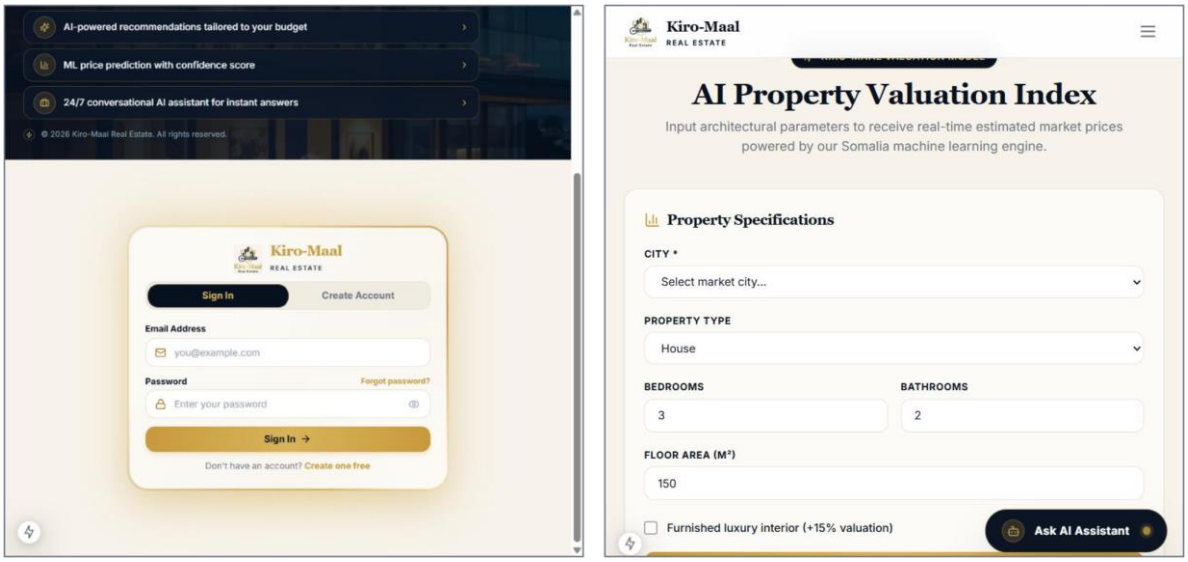


Figure 4.4: Login Form and Price Prediction Form

Table 4.18: Main Forms of the System

Form	User	Main Fields
Registration	Customer	Name, e-mail address, telephone number, password
Login	Registered actors	E-mail address, password
Property	Property Manager	Title, description, listing type, category, price or rent, deposit, images, city, address, amenities
Rental booking	Customer	Rental period, start and end dates, payment method, notes
Direct rental	Property Manager	Property, customer, dates, rent, security deposit, payment status, lease clauses
Purchase request	Customer	Property, notes to the administrator
Payment	Customer	Payment method and confirmation of the amount
Inquiry	Public Visitor, Customer	Name, contact, subject, message
Price prediction	Public Visitor, Customer	City, property type, bedrooms, bathrooms, floor area, furnished
Add customer	Property Manager	Full name, e-mail address, telephone number, optional password

The reports of the system are the role dashboards, the lists of properties, customers, rentals and requests, the central transaction ledger, the analytics of the Administrator and the digital receipt. Lists are presented as tables that can be searched, filtered and sorted, and a status is always shown as a labelled badge. The receipt is designed as a printable document that can be exported as PDF. Figure 4.5 shows the layout designed for the dashboards and for the receipt.

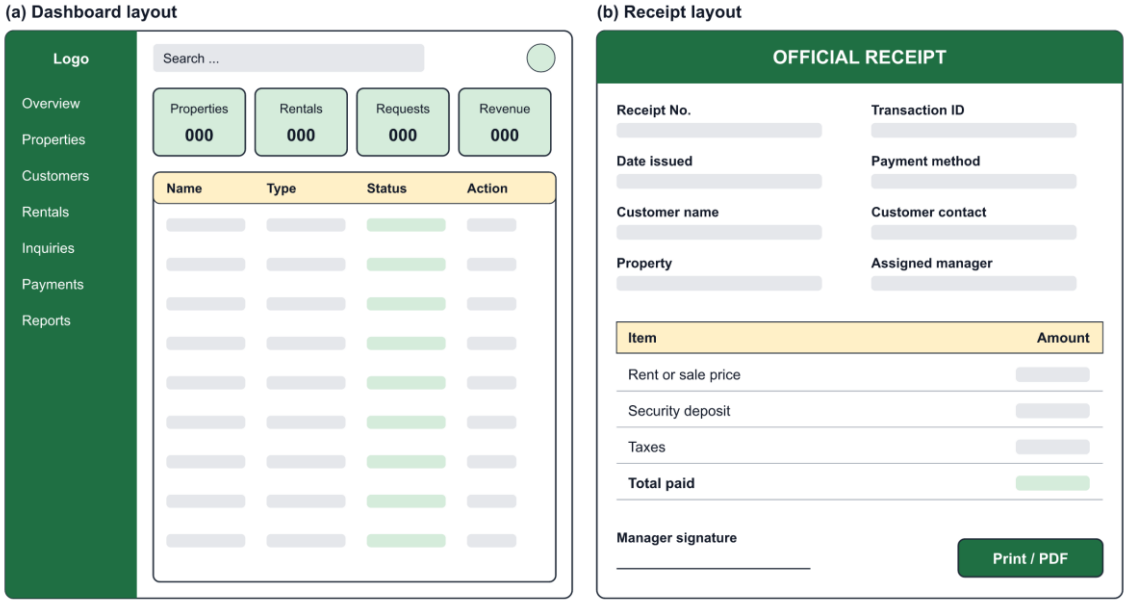


Figure 4.5: Layout Design of the Dashboards and the Receipt

Table 4.19: Main Reports of the System

Report	User	Content
Customer dashboard	Customer	Active leases, pending purchases, saved properties, inquiries
Manager overview	Property Manager	Counts of properties and requests, today's appointments, recent activity
Administrator analytics	Administrator	Transaction value, revenue, active properties, leases, user growth
Transaction ledger	Manager, Administrator	Every deposit, rent payment and purchase, with method and status
Customer directory	Manager, Administrator	Customers with their leases, purchase requests and payments
Receipt	Customer	Itemized amounts, transaction number, customer and manager details
Audit log	Administrator	Administrative actions with the user and the time

4.6 Chapter Summary

This chapter presented the design of the system. The architectural design organized the system into presentation, application and data layers, and the user interface design defined the

navigation of the four actors and the principles of the interface. The database storage design and the database design described how data are stored in MySQL, normalized the data to third normal form, transformed the ERD into relations and defined the data dictionaries of the central tables. Finally, the main forms and reports of the system were designed and listed. The next chapter describes how this design was implemented and tested.